

Zaadi Kitchen

High-Level Architecture & Technical Design Document

Flutter · NestJS · React · AWS

April 2025 | Version 1.0 | Confidential

1. System Overview

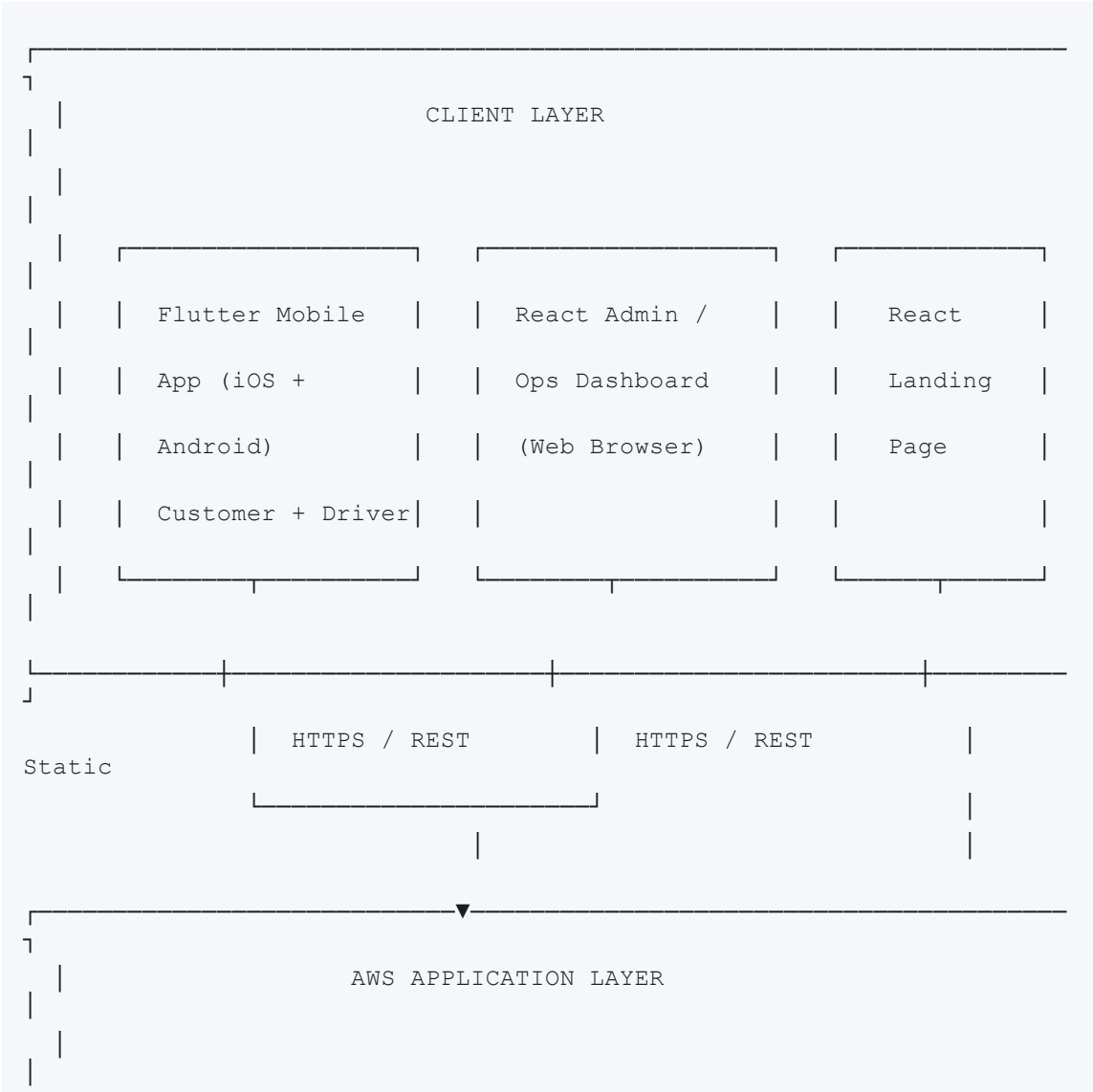
Zaadi Kitchen is a B2C meal plan subscription platform serving office workers in GCC markets. Users subscribe to a meal plan (weekly, monthly, or quarterly), receive a freshly prepared meal delivered to their desk each weekday, and manage every aspect of their subscription — skipping days, pausing, switching meal types, and renewing — through the mobile app.

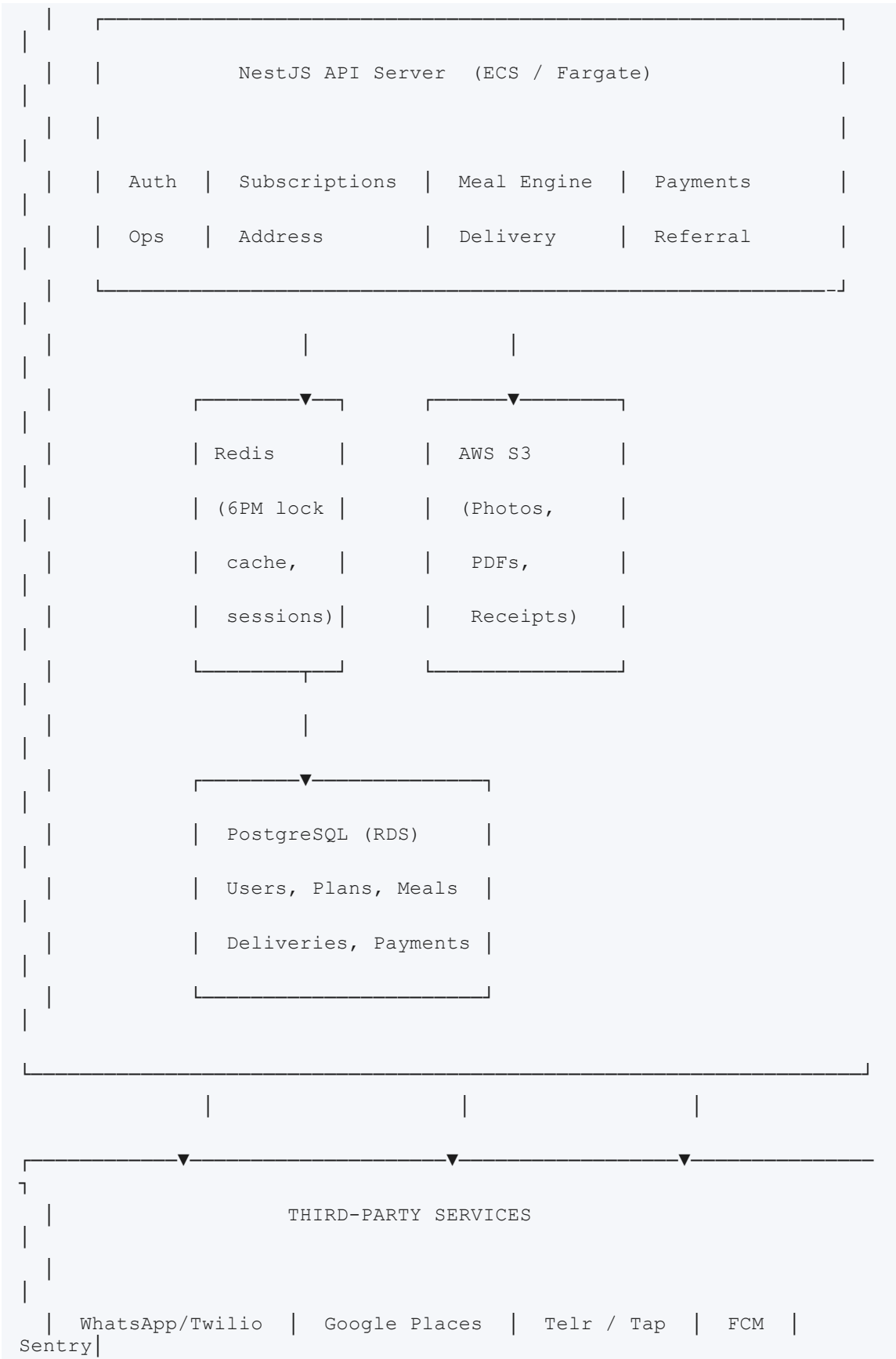
The platform consists of five integrated components that communicate over HTTPS REST APIs:

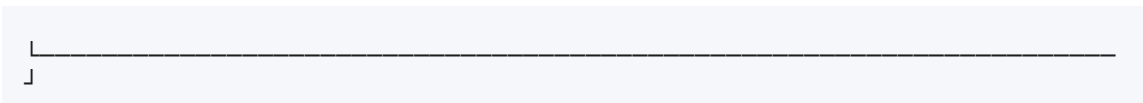
- Customer + Driver Mobile App (Flutter) — single codebase, dual role via server-side flag
- Backend API Server (NestJS / Node.js) — stateless REST, subscription state machine, meal engine
- Admin & Ops Dashboard (React) — kitchen operations, label printing, support, analytics
- Landing Page (React) — minimal static marketing page
- Cloud Infrastructure (AWS) — managed database, cache, storage, CDN, container hosting

1.1 High-Level Architecture

The following diagram shows how the five components interact at a system level.







1.2 Request Flow Summary

All mobile and web requests flow through the NestJS API. There is no direct client-to-database or client-to-cache connection. Third-party integrations (payments, OTP, push) are always proxied through the backend.

From	To	Purpose
Mobile App	→ NestJS API → PostgreSQL	Standard CRUD, plan management, meal data
Mobile App	→ NestJS API → Redis	6 PM lock check, session validation, rate limit
Mobile App	→ NestJS API → S3	Photo uploads (rating, driver issues) via presigned URL
Admin Dashboard	→ NestJS API → PostgreSQL + S3	Ops data reads, label PDF generation
Payment Gateway	→ NestJS Webhook → PostgreSQL	Async payment confirmation, subscription activation
NestJS Cron (18:00)	→ Redis	Daily 6 PM lock job — freezes skip/switch for next day
NestJS	→ WhatsApp / Twilio	OTP dispatch on login and sign-up
NestJS	→ FCM	Push notification dispatch — delivery alerts, reminders

2. Component Design

2.1 Mobile App (Flutter)

A single Flutter codebase targets both iOS and Android. Role-based routing (CUSTOMER vs DRIVER) is determined server-side from the OTP verify response, so no separate driver app is needed.

Navigation Architecture

Stack	Screens	Condition
Unauthenticated stack	Splash → Language → Phone → OTP → Sign-up (3 screens)	Pre-login flow, no JWT required
Discovery stack	Discovery Home → Plan Selection → Address Setup	First-time user, no active plan
Active subscriber stack	Home → Menu → My Plan → Skip → Pause → Profile	Core subscription management
Renewal stack	Home (expired) → Renew → Checkout → Gateway → Success	Re-engagement flow
Driver stack	Delivery List → Detail → QR Scan	Separate nav stack, role-gated

State Management

Use Riverpod (or BLoC) for state. The subscription state enum drives the entire home screen presentation — a single SubscriptionState provider determines which home layout renders.

- SubscriptionState enum: NEW | ACTIVE | PAUSED | EXPIRED
- AuthState: UNAUTHENTICATED | AUTHENTICATED (with role: CUSTOMER | DRIVER)
- MenuState: loaded per week, cached locally for offline viewing
- CartState: in-memory only, cleared on app restart

Offline & Caching Strategy

- Weekly menu cached to local storage on first load, refreshed Monday morning
- User profile and plan status cached, invalidated on every app foreground
- All mutation operations (skip, pause, switch) require network — no offline queue

2.2 Backend API (NestJS)

NestJS provides a modular, decorator-based architecture that maps cleanly to the domain modules of this platform. Each module owns its routes, services, and database entities.

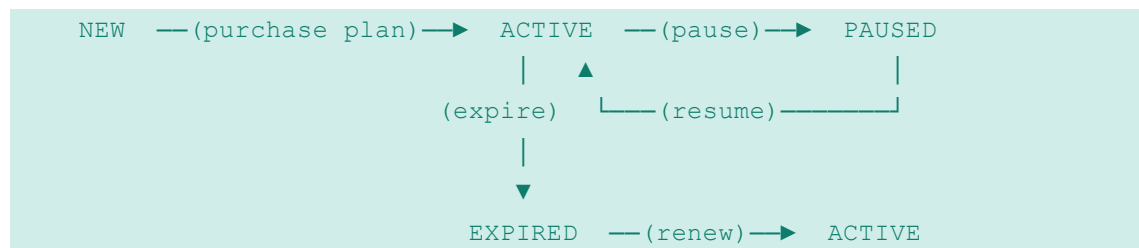
Module Structure

Module	Responsibilities
AuthModule	OTP generation/verify, JWT issue/refresh, role guards, device token management

Module	Responsibilities
UserModule	Profile CRUD, address management, language preference
SubscriptionModule	Plan state machine, create/renew/pause/resume, days-remaining calculation
MealModule	Weekly menu, meal details, switch-type, skip-day, undo-skip, salad toggle
DeliveryModule	Daily delivery records, status updates, driver assignment, issue reports
PaymentModule	Gateway session init, webhook handler, billing history, wallet ledger
ReferralModule	Code generation, application, credit tracking
AdminModule	Production counts, label payload, customer support actions, analytics aggregates
NotificationModule	FCM dispatch, WhatsApp messaging, notification preferences

Subscription State Machine

This is the most critical piece of backend logic. The state machine governs every screen transition in the mobile app.



- NEW: Account created, no active subscription. Shows Discovery home.
- ACTIVE: Paid, current plan running. Full app access. Skip and Switch enabled (subject to 6 PM lock).
- PAUSED: User-requested pause within allowed date range. Home shows resume CTA. No deliveries generated.
- EXPIRED: Plan end date passed. Home shows renewal CTA only. Skip, switch, and pause are locked.

6 PM Lock — Critical Implementation Detail

⚠ Critical The 6 PM lock freezes all skip and meal-type switch operations for the next calendar day's delivery. It must be enforced server-side regardless of client state.

- A Redis cron job runs at 18:00 local timezone daily
- It writes a lock key: lock:delivery:{YYYY-MM-DD} with TTL = 30 hours
- All skip-day and switch-meal-type API endpoints check this Redis key before processing
- The mobile app reads a can_modify field in the meal response and disables UI accordingly
- An alerting rule fires if the cron has not executed within 5 minutes of 18:00

Database Design — Core Entities

PostgreSQL via Prisma ORM. All tables use UUID primary keys.

users

Field	Type	Nullable	Description
id	uuid	No	Primary key
phone	varchar(20)	No	Unique, used as login identifier
name	varchar(100)	Yes	Display name
email	varchar(150)	Yes	Optional
language	varchar(5)	No	'en' or 'ar', default 'en'
role	enum	No	CUSTOMER DRIVER ADMIN
fcm_token	text	Yes	Firestore push token, updated on login
referral_code	varchar(12)	No	Unique auto-generated on sign-up
wallet_balance	decimal(10,2)	No	In-app credit balance, default 0
created_at	timestamp	No	

subscriptions

Field	Type	Nullable	Description
id	uuid	No	Primary key
user_id	uuid	No	FK → users.id
plan_type	enum	No	TRY_IT WEEKLY MONTHLY QUARTERLY
state	enum	No	NEW ACTIVE PAUSED EXPIRED
meal_type	enum	No	STANDARD DIET LARGE
start_date	date	No	First delivery date
end_date	date	No	Last delivery date (recalculated on pause/resume)
skip_allowance	int	No	Max skips permitted for this plan
skips_used	int	No	Current skip count
paused_at	timestamp	Yes	When pause was requested
resume_at	date	Yes	Scheduled resume date
address_id	uuid	No	FK → addresses.id — active delivery address

deliveries

Field	Type	Nullable	Description
id	uuid	No	Primary key
subscription_id	uuid	No	FK → subscriptions.id
meal_id	uuid	No	FK → meals.id
delivery_date	date	No	Scheduled delivery date
status	enum	No	SCHEDULED SKIPPED OUT_FOR_DELIVERY DELIVERED ISSUE
driver_id	uuid	Yes	FK → users.id (driver role)
delivered_at	timestamp	Yes	Actual delivery timestamp
has_salad	boolean	No	Salad add-on for this delivery, default false
driver_notes	text	Yes	Issue description if status = ISSUE

payments

Field	Type	Nullable	Description
id	uuid	No	Primary key
user_id	uuid	No	FK → users.id
subscription_id	uuid	Yes	FK → subscriptions.id (null for wallet top-ups)
gateway	enum	No	TELR TAP
gateway_ref	varchar(100)	No	External transaction reference
amount	decimal(10,2)	No	Amount charged in local currency
currency	varchar(3)	No	ISO 4217 — AED, SAR, KWD etc.
status	enum	No	PENDING SUCCESS FAILED REFUNDED
idempotency_key	varchar(64)	No	Prevents duplicate webhook processing
created_at	timestamp	No	

2.3 Admin Dashboard (React)

Built with React + TypeScript, using a minimal component library (shadcn/ui or Ant Design). The dashboard is optimised for daily ops use — fast load, keyboard-friendly, no unnecessary decoration.

Module	Purpose	Used By
Daily Ops	Live meal count by type, production lock status (Locked at 6 PM / Cooking)	Kitchen team — daily
Label Generator	Per-building PDF label batch at 100×60 mm, select all or per building	Kitchen team — daily
Driver Export	CSV per building sorted by floor and desk for driver briefing	Ops team — daily
Menu Management	CRUD for weekly meals — name, photo, calories, type, salad flag	Admin — weekly
Customer List	Search and filter subscribers by state, plan, building	Support — ad hoc
Customer Detail	Full plan history, skips, pauses, ratings, address, wallet	Support — ad hoc
Issue Queue	Driver + customer reported issues, resolve/credit actions	Support — daily
Revenue View	MRR, new/churned counts, plan distribution, referral stats	Management — weekly

3. Third-Party Services

The following external services are required to operate the platform. All must be contracted and configured before launch. Costs marked as 'per transaction' or 'per message' scale with usage.

Service	Provider	Purpose	Priority	Cost Model
WhatsApp Business API	Meta / Twilio	OTP delivery	Required	message credits per OTP
SMS Gateway	Twilio	OTP fallback	Required	per SMS, ~\$0.05–0.10
Google Places API	Google Cloud	Building search	Required	per request, cache heavily
Firebase Cloud Messaging	Google Firebase	Push notifications	Required	Free up to high volume
Telr Payment Gateway	Telr	Card, Apple Pay	Option A	% per transaction
Tap Payments	Tap	Card, Apple Pay, KNET	Option B	% per transaction
AWS S3	Amazon Web Services	File storage	Required	per GB stored + egress
AWS RDS (PostgreSQL)	Amazon Web Services	Primary database	Required	instance + storage/hr
AWS ElastiCache (Redis)	Amazon Web Services	Cron, cache, sessions	Required	node instance/hr
AWS CloudFront	Amazon Web Services	CDN for assets	Required	per GB transferred
AWS ECS / Fargate	Amazon Web Services	Container hosting	Required	per vCPU + memory/hr
Firebase Analytics	Google Firebase	App analytics	Recommended	Free
Sentry	Sentry.io	Error monitoring	Recommended	Free tier available
Apple Developer	Apple	iOS App Store	Required	\$99 USD/yr
Google Play Developer	Google	Android Play Store	Required	\$25 USD one-time

 **Note** Google Places API and WhatsApp Business API costs can be significant at scale. The Places API should be aggressively cached — store building results in the database after first lookup and serve from cache for subsequent searches.

3.1 WhatsApp OTP vs SMS Fallback

The primary OTP channel is WhatsApp Business API (higher open rates in GCC). SMS via Twilio is the automatic fallback if WhatsApp delivery fails within 15 seconds.

- WhatsApp delivery: typical < 3 seconds in GCC
- SMS fallback triggers: delivery failure, number not on WhatsApp, user-initiated 'resend via SMS'
- Both channels rate-limited to 5 attempts per phone number per 10 minutes

3.2 Payment Gateway — Telr vs Tap

Feature	Telr	Tap Payments
Apple Pay	Yes	Yes
Google Pay	Yes	Yes
VISA / Mastercard	Yes	Yes
KNET (Kuwait)	No	Yes
MADA (Saudi)	Yes	Yes
Recurring / Tokenised	Yes — Card Vault	Yes — Token API
Webhook reliability	High	High
Dashboard quality	Good	Excellent
GCC coverage	UAE, SA, BH, OM, QA	UAE, SA, KW, BH, QA
Recommended for	UAE-first launch	Multi-market from day 1

4. API Documentation

Base URL: <https://api.zaadikitchen.com/v1>

All endpoints require Content-Type: application/json. Authenticated endpoints require Bearer token in Authorization header unless marked Public.

Auth levels used in tables below:

- Public — no token required
- JWT — valid customer/driver JWT required
- Admin — valid admin JWT required
- Driver — valid driver JWT required

4.1 Authentication

Method	Endpoint	Auth	Description	Request (key fields)	Response
POST	/auth/send-otp	Public	Send OTP to phone via WhatsApp / SMS	phone, channel?	{ message_id }
POST	/auth/verify-otp	Public	Verify OTP, return JWT tokens	phone, otp	{ access_token, refresh_token, user, is_new_user }
POST	/auth/refresh	Public	Refresh access token	refresh_token	{ access_token, refresh_token }
POST	/auth/logout	JWT	Invalidate refresh token	—	{ success: true }

4.2 User & Address

Method	Endpoint	Auth	Description	Request (key fields)	Response
GET	/users/me	JWT	Get current user profile	—	{ user object }
PATCH	/users/me	JWT	Update profile fields	name?, email?, language?, fcm_token?	{ user object }
GET	/users/me/addresses	JWT	List all saved addresses	—	[address[]]
POST	/users/me/addresses	JWT	Save a new address	building_id, floor,	{ address object }

Method	Endpoint	Auth	Description	Request (key fields)	Response
				desk, notes?	
PATCH	/users/me/addresses/:id	JWT	Update an address	floor?, desk?, notes?	{ address object }
DELETE	/users/me/addresses/:id	JWT	Delete an address	—	{ success: true }
POST	/users/me/addresses/:id/default	JWT	Set address as default	—	{ success: true }
GET	/buildings/search	JWT	Search buildings via Places API	q (query string)	[{ id, name, address }]

4.3 Plans & Subscriptions

Method	Endpoint	Auth	Description	Request (key fields)	Response
GET	/plans	Public	List available plan tiers with pricing	—	[plan[]]
GET	/subscriptions/me	JWT	Get current subscription status	—	{ state, plan_type, days_remaining, skips_used, skip_allowance, can_skip_to_day, can_switch_today, meal_type, address }
POST	/subscriptions	JWT	Create new subscription (first purchase)	plan_type, meal_type, address_id, promo_code?	{ subscription, payment_session_url }
POST	/subscriptions/me/renew	JWT	Renew or upgrade current plan	plan_type, meal_type?, address_id?	{ subscription, payment_session_url }
POST	/subscriptions/me/pause	JWT	Pause subscription	pause_from, pause_until	{ subscription }
POST	/subscriptions/me/resume	JWT	Resume a paused subscription	resume_from?	{ subscription }
PATCH	/subscriptions/me/meal-type	JWT	Switch meal type	meal_type, scope (ALL	{ subscription }

Method	Endpoint	Auth	Description	Request (key fields)	Response
				SPECIFIC), dates?[]	

4.4 Meal Engine

Method	Endpoint	Auth	Description	Request (key fields)	Response
GET	/meals/menu	JWT	Get weekly menu (Mon–Fri)	week? (ISO date of Monday)	{ week, days: [{ date, meals[] }] }
GET	/meals/:id	JWT	Get meal detail	–	{ id, name, description, calories, macros, allergens, has_salad_option }
POST	/deliveries/:id/skip	JWT	Skip a delivery	–	{ delivery, skips_remaining }
DELETE	/deliveries/:id/skip	JWT	Undo a skip	–	{ delivery, skips_remaining }
PATCH	/deliveries/:id/salad	JWT	Toggle salad add-on	has_salad (boolean)	{ delivery }
GET	/deliveries/history	JWT	Paginated meal history	page?, limit?, from?, to?	{ data: delivery[], total, calories_total }

4.5 Payments

Method	Endpoint	Auth	Description	Request (key fields)	Response
POST	/payments/session	JWT	Initiate payment session	plan_type, amount, currency, wallet_apply?	{ session_url, session_id }
POST	/payments/webhook/telr	Public	Telr payment webhook	Telr payload (signature verified)	HTTP 200
POST	/payments/webhook/tap	Public	Tap payment webhook	Tap payload (signature verified)	HTTP 200

Method	Endpoint	Auth	Description	Request (key fields)	Response
GET	/payments/history	JWT	Get billing history	page?, limit?	{ data: payment[], total }
GET	/payments/:id/receipt	JWT	Download receipt PDF	—	PDF file stream
GET	/users/me/wallet	JWT	Get wallet balance	—	{ balance, currency, transactions[] }

4.6 Delivery & Driver

Method	Endpoint	Auth	Description	Request (key fields)	Response
GET	/driver/deliveries	Driver	Get today's delivery list	—	[{ delivery_id, address, meal, status, floor, desk }]
PATCH	/driver/deliveries/:id/status	Driver	Update delivery status	status (DELIVERED ISSUE)	{ delivery }
POST	/driver/deliveries/:id/issue	Driver	Report a delivery issue	category, description, photo_url?	{ issue }
POST	/driver/deliveries/:id/verify	Driver	Verify delivery via QR	qr_token	{ success, delivery }

4.7 Ratings & Referral

Method	Endpoint	Auth	Description	Request (key fields)	Response
POST	/ratings	JWT	Submit meal rating	delivery_id, stars (1-5), note?, photo?	{ rating }
GET	/referral/me	JWT	Get referral code and stats	—	{ code, uses, credits_earned }
POST	/referral/apply	JWT	Apply referral code at sign-up	code	{ credited_amount }

Method	Endpoint	Auth	Description	Request (key fields)	Response
POST	/media/upload-url	JWT	Get presigned S3 upload URL	file_type, purpose	{ upload_url, file_url }

4.8 Admin Endpoints

Method	Endpoint	Auth	Description	Request (key fields)	Response
GET	/admin/ops/daily-production	Admin	Meal counts by type for today	date?	{ date, counts: { STANDARD, DIET, LARGE, total } }
GET	/admin/ops/labels	Admin	Label data for PDF generation	date?	[{ order_id, name, meal, address, floor, desk }]
GET	/admin/ops/deliveries	Admin	Full delivery board for date	date?	[delivery[] with status]
GET	/admin/customers	Admin	Paginated customer list	state?, plan?, building_id?, search?, page?	{ data: user[], total }
GET	/admin/customers/:id	Admin	Full customer detail	—	{ user, subscription, deliveries[], payments[], ratings[] }
POST	/admin/customers/:id/credit	Admin	Apply wallet credit	amount, note	{ wallet_balance }
GET	/admin/issues	Admin	Issue queue	status?, from?, to?	[issue[]]
PATCH	/admin/issues/:id	Admin	Resolve or close issue	status, resolution_note	{ issue }
GET	/admin/analytics/revenue	Admin	MRR and subscriber stats	from?, to?	{ mrr, new_subscribers, churned, active, plan_distribution }

Method	Endpoint	Auth	Description	Request (key fields)	Response
GET	/admin/menu	Admin	Get weekly menu for editing	week?	{ week, meals[] }
POST	/admin/menu/meals	Admin	Create meal	name, description, calories, macros, meal_type, has_salad	{ meal }
PATCH	/admin/menu/meals/:id	Admin	Update meal	any meal fields	{ meal }
DELETE	/admin/menu/meals/:id	Admin	Delete meal	—	{ success: true }

5. Security Design

5.1 Authentication Flow

The platform uses short-lived JWTs (15 minutes) with rotating refresh tokens (30 days). OTP is the sole authentication mechanism — no passwords.

1. User enters phone → POST /auth/send-otp Server generates 6-digit OTP, stores in Redis with TTL=300s Dispatches OTP via WhatsApp (primary) or SMS (fallback)
2. User enters OTP → POST /auth/verify-otp Server validates OTP from Redis, deletes key on success Returns: access_token (15 min JWT) + refresh_token (30 day, stored in DB)
3. Client stores both tokens securely (Flutter Secure Storage) access_token sent as Bearer on every API request
4. When access_token expires → POST /auth/refresh Server validates refresh_token, issues new token pair Old refresh_token is invalidated (rotation prevents replay)
5. Logout → POST /auth/logout Server deletes refresh_token from DB Client clears both tokens from secure storage

5.2 Security Checklist

Risk / Control	Implementation
OTP brute force	Max 5 attempts per phone per 10 min. Account locked for 30 min after 3 failed windows.
JWT secret rotation	JWT_SECRET stored in AWS Secrets Manager. Rotation does not require redeployment.
Webhook signature verification	Both Telr and Tap webhooks verified against HMAC signature before processing.
Idempotency on payments	Every webhook event has an idempotency_key. Duplicate events are silently ignored.
SQL injection	Prisma ORM with parameterised queries. No raw SQL in application code.
S3 access	All files accessed via presigned URLs (15 min TTL). Bucket is not public.
Admin route protection	Separate JWT claim (role: ADMIN) + IP allowlist for admin dashboard domain.

Risk / Control	Implementation
HTTPS everywhere	All API and static traffic served over TLS 1.2+. HTTP redirects to HTTPS.
Sensitive data	Phone numbers and wallet balances encrypted at rest in RDS.
Rate limiting	NestJS ThrottlerModule: 100 req/min per IP on public routes, 300 req/min per user on authenticated routes.

6. AWS Infrastructure Design

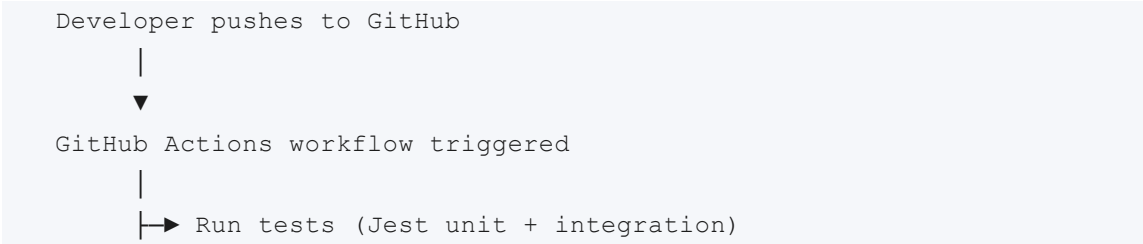
6.1 Services Used

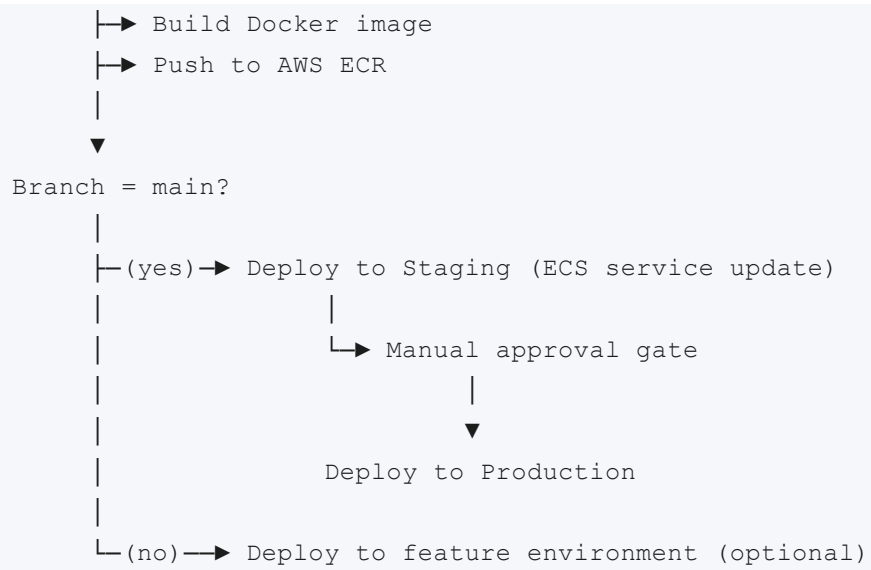
Service	Usage
ECS / Fargate	Containerised NestJS API. Auto-scales on CPU/memory. No server management required.
RDS PostgreSQL	Multi-AZ deployment for production. Automated daily snapshots with 7-day retention.
ElastiCache Redis	Single-node for staging, 2-node cluster for production. Used for 6 PM lock cron, session store, rate limiting.
S3	Three buckets: media (photos, not public), exports (labels, receipts, temp presigned), static (landing page assets).
CloudFront	CDN in front of S3 static bucket and API. Reduces latency for GCC users.
Secrets Manager	All credentials: DB password, JWT secret, gateway API keys, WhatsApp token.
CloudWatch	Application logs, API latency metrics, cron job execution alerts, error rate dashboards.
ACM	SSL certificates auto-renewed for api.zaadikitchen.com and app.zaadikitchen.com.
Route 53	DNS for all subdomains. Health checks on the API endpoint.

6.2 Environment Strategy

Environment	Setup	Purpose
Development	Local Docker Compose — NestJS + PostgreSQL + Redis	Feature development, daily use
Staging	Single Fargate task, single RDS instance, shared Redis	Integration testing, UAT
Production	Fargate with min 2 tasks, Multi-AZ RDS, Redis cluster	Live traffic

6.3 CI/CD Pipeline





7. Key Business Flow Walkthroughs

7.1 New User → First Subscription Purchase

1. User opens app → Splash → Language selection
2. Phone number entry → POST /auth/send-otp → WhatsApp OTP sent
3. OTP entry → POST /auth/verify-otp → JWT issued, is_new_user: true
4. Sign-up screens (3): basic info, building search, desk details → PATCH /users/me + POST /addresses
5. Discovery home displayed (subscription state = NEW)
6. User taps 'Choose Plan' → Plan selection screen → selects Monthly
7. Checkout → POST /subscriptions → server creates PENDING subscription, returns payment session URL
8. Payment gateway screen → user completes Apple Pay
9. Gateway fires webhook → POST /payments/webhook/telr → server verifies, marks payment SUCCESS, transitions subscription to ACTIVE
10. App polls GET /subscriptions/me → state = ACTIVE → navigates to Active home screen

7.2 Skip a Day (Happy Path)

11. User opens app before 18:00. GET /subscriptions/me returns can_skip_today: true
12. Home screen shows 'Quick Skip' button enabled
13. User taps Skip → Skip a Day screen → selects tomorrow's date
14. POST /deliveries/{id}/skip → server checks Redis lock key (not set) → decrements skips_used → sets delivery status = SKIPPED
15. Success toast shown. Calendar updates to show skip. Undo button visible.
16. At 18:00 → cron fires → Redis key lock:delivery:{tomorrow} written
17. Any skip request after 18:00 → server returns 423 Locked → app shows 'Skip window closed for today'

7.3 Pause → Resume

18. User opens My Plan → taps Pause
19. Pause screen: user selects pause_from and pause_until dates (max 14 days, configurable)
20. POST /subscriptions/me/pause → server validates dates, transitions state to PAUSED, updates end_date to extend by pause duration
21. All deliveries in pause range set to status = SKIPPED (no driver assigned, no production count)
22. Home screen shows paused banner with resume date and Resume CTA
23. On resume_at date → scheduled job transitions state back to ACTIVE automatically
24. Or user taps Resume early → POST /subscriptions/me/resume → immediate ACTIVE transition

7.4 Driver Daily Flow

25. Driver logs in with phone (same OTP flow, role = DRIVER returned in JWT)

26. Driver home = Delivery List screen. GET /driver/deliveries returns today's list sorted by building → floor → desk
27. Driver taps an order → Delivery Detail screen
28. Driver confirms delivery → PATCH /driver/deliveries/{id}/status { status: DELIVERED } → timestamp recorded
29. Alternatively: Driver scans QR on the label → POST /driver/deliveries/{id}/verify { qr_token } → same status update
30. If issue: Driver taps 'Report Issue' → selects category → POST /driver/deliveries/{id}/issue → issue appears in admin queue

8. Environment Variables Reference

All secrets stored in AWS Secrets Manager. Non-secret config stored in ECS task definition environment variables.

Variable	Description / Notes
DATABASE_URL	PostgreSQL connection string (from Secrets Manager)
REDIS_URL	ElastiCache Redis connection string
JWT_SECRET	JWT signing secret (rotate every 90 days via Secrets Manager)
JWT_EXPIRES_IN	Access token TTL, default '15m'
REFRESH_TOKEN_EXPIRES_IN	Refresh token TTL, default '30d'
TWILIO_ACCOUNT_SID	Twilio account identifier
TWILIO_AUTH_TOKEN	Twilio auth token (Secrets Manager)
TWILIO_PHONE_NUMBER	SMS sender number
WHATSAPP_API_URL	WhatsApp Business API base URL
WHATSAPP_TOKEN	WhatsApp API bearer token (Secrets Manager)
WHATSAPP_PHONE_ID	WhatsApp Business phone number ID
GOOGLE_PLACES_API_KEY	Places API key (Secrets Manager)
TELR_STORE_ID	Telr merchant store ID
TELR_AUTH_KEY	Telr auth key (Secrets Manager)
TAP_SECRET_KEY	Tap Payments secret key (Secrets Manager)
AWS_S3_BUCKET_MEDIA	S3 bucket name for media uploads
AWS_S3_BUCKET_EXPORTS	S3 bucket name for PDF exports
AWS_REGION	AWS region, e.g. me-south-1 (Bahrain) or eu-west-1
FCM_SERVICE_ACCOUNT_JSON	Firebase service account JSON (Secrets Manager)
LOCK_CRON_TIMEZONE	Timezone for 6 PM lock cron, e.g. Asia/Dubai
PUPPETEER_EXECUTABLE_PATH	Chromium path inside Docker container for PDF generation
SENTRY_DSN	Sentry error reporting DSN
NODE_ENV	development staging production

9. Non-Functional Requirements

Requirement	Target	Implementation Note
API Response Time	P95 < 300ms for read endpoints, < 800ms for mutation endpoints	Measured at CloudFront edge
API Uptime	99.5% monthly uptime target	Multi-AZ RDS + ECS min 2 tasks in production
Mobile App Launch	Cold start < 3 seconds on mid-range Android (2019 or newer)	Flutter AOT compilation
6 PM Cron Reliability	Must fire within ± 5 minutes of 18:00 daily	CloudWatch alarm if missed
Concurrent Users	Support 500 concurrent active sessions at launch	Fargate auto-scaling policy
Data Retention	User data retained for 3 years, delivery history for 5 years	RDS snapshot + S3 archival
Payment Webhook	Process and acknowledge within 5 seconds	Idempotency + async processing
App Bundle Size	< 30 MB download size (Flutter release build)	Tree-shaking, no unused assets
Localisation	Full EN/AR support, RTL layout, Arabic numerals in date fields	Flutter Intl package

Document Information

This document covers the high-level architecture, component design, API specification, third-party service requirements, and key business flows for Zaadi Kitchen v1.0.

Prepared for: Zaadi Kitchen

Document version: 1.0 | April 2025

This document should be reviewed alongside the Project Estimation document for complete scope and effort context.